

Engineering Journal

Team ID: FE-XXX-01

Team name: Lockedin Martin

Team members:

- Yuan Hao Zhe
- Liu Xinji
- Om Joshi vaibhav

BOM

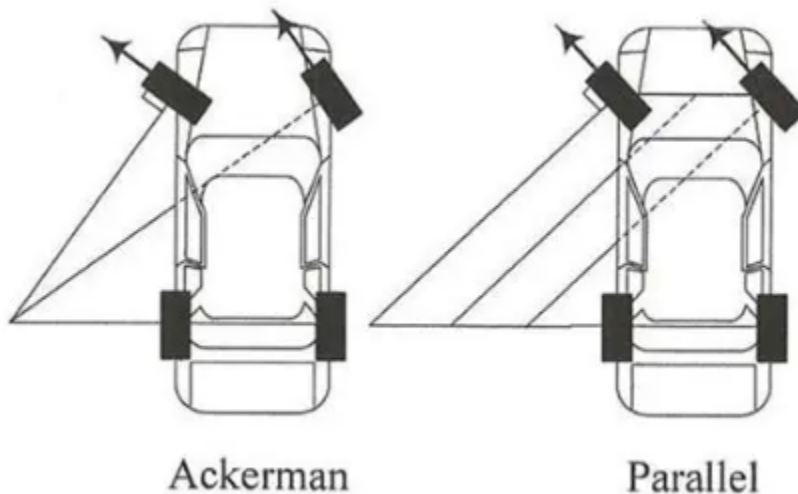
Component type	component	quantity
1x Main MCU	Openmv H7 plus	https://e.tb.cn/h.8k547RCXRMKPPMn?tk=YBIXTWmNGnR CA381
1x Sensor reading MCU	Waveshare RP2040 zero	https://e.tb.cn/h.8PjmAA171s06Ows?tk=IHOLTWmmZNu HU591
2x TOF sensor	VL53L1X	https://e.tb.cn/h.8IDQ4xvxufmwdAn?tk=JVRuTWmoiVx MF937
1x Motor driver	DRV8874	https://e.tb.cn/h.8PjKGGh9RlypMOC?tk=HPOsTWmoE9O CZ001 「D
1x IMU	BNO085	https://kuriosity.sg/products/9dof-9-axis-accelerometer-gyroscope-magnetic-field-sensor-bno085?_pos=1&_sid=b334db9a9&_ss=r
1x motor	1020 coreless motor	https://e.tb.cn/h.8955D6mjnewGMLc?tk=FXHUTWmpd1r MF278
4x wheels	POLOLU 34mm silicone wheels	https://e.tb.cn/h.8IDInlgMV7zfWEh?tk=kgliTWmKsee CZ005
1x servo	SG90 servo	https://kuriosity.sg/products/servo-motor-sg92r-180-36

		0-sg90-upgrade?_pos=1&_sid=28d788176&_ss=r
1x battery	TATTU R-LINE 2S 500MAH battery	https://hobbysquare.com.sg/collections/battery-2s-7-4v/products/tattu-r-line-7-4v-2s-550mah-95c-lipo-micro-battery-xt30
1x Buck converter	5v buck converter	https://kuriosity.sg/products/dc-dc-step-down-buck-converter-3-3v-5v-12v-3a?variant=51054170308921
1x Buck converter	3.3v buck converter	https://kuriosity.sg/products/dc-dc-step-down-buck-converter-3-3v-5v-12v-3a?_pos=3&_sid=1854b2fae&_ss=r
1x Battery lead	XT30 female lead	https://hobbysquare.com.sg/products/xt30-connector-male-and-female-2-pairs?_pos=4&_sid=2305c2fce&_ss=r
4x bearing	2mm ID, 5mm OD, 2.5mm thick bearing	https://e.tb.cn/h.8k5j7aNDYUkgbZ3?tk=3Qu4TWmt2iLHU926
1x Motor gear	10 tooth gear, 0.5mm ID	https://e.tb.cn/h.8k5SP1IUNwkpewx?tk=18RbTWmrFV0MF278
1x Shaft gear	30 tooth gear, 2mm ID	https://e.tb.cn/h.8957CbRobZdnOfa?tk=G1rCTWmsS3bMF937Γ0
2x Metal shaft	2mm by 12mm stainless steel rod	https://e.tb.cn/h.895RU63o9vwlXqi?tk=8JWuTWmH8mV MF168

Mechanical design

Steering mechanism selection

- In an ideal steering mechanism, the path of all the wheels trace out circles that centres on the same point, which is the instantaneous centre of rotation (ICR) of the vehicle. This ensures that all wheels are tangential to the circular path that they travel, hence preventing lateral tire slippage. This reduces vibration and ensuring that the turning behaviour of the vehicle remains predictable
- A neutral steering mechanism has both front wheels being parallel to one another, while an ackermann steering mechanism has the inner wheel (the one closer to the ICR) turning more than the outer wheel. A reverse ackermann steering mechanism has the outer wheel turning more than the inner wheel.
- When a vehicle with a neutral steering mechanism turns rightwards, the centre of the circle that the inner (right) wheel traces lies to the right of that which the outer (left) wheel traces, resulting in lateral wheel slippage.
- An ackermann steering assembly remedies this problem by making sure the inner wheel turns more than the outer wheel. A reverse ackermann mechanism exacerbates this problem

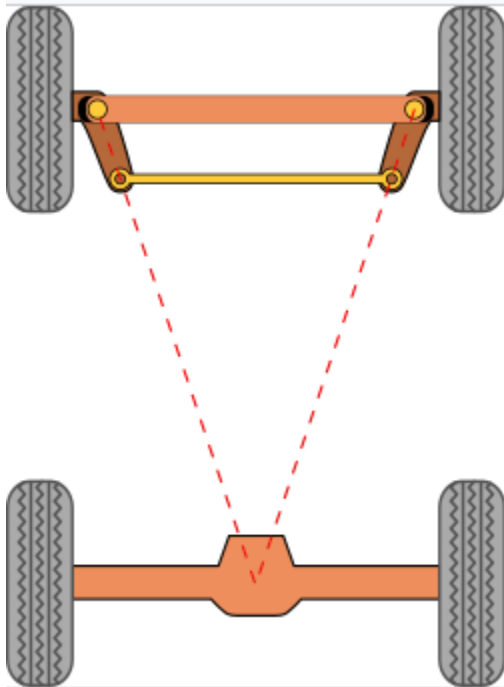


- A neutral or reverse ackermann mechanism offers different advantages. During high-speed cornering, weight shifts dramatically to the outer front tire. The outer tire generates the vast majority of cornering force and operates at a higher slip angle, while the unloaded inner tire contributes very little. Turning the outer tire more aggressively using a reverse ackermann mechanism maximizes peak cornering grip, improving high speed cornering performance.

- *For our application, where precise and predictable low speed control is necessary to navigate around obstacles and to parallel park, the ideal steering mechanism would be the ackermann steering mechanism.*

Ackermann mechanism key dimensions

In a standard ackermann steering mechanism, the front wheels are mounted to a steering rods that are connected by a steering linkage. The steering rods are angled in such a way that lines drawn through them intersect in the middle of the rear axle.



Motor and wheel sizing (from our LATEX document)

1 motor and wheel sizing

The torque required to spin the wheel is given by $\tau_{wheel} = I_{wheel}\alpha$ where I_{wheel} is the moment of inertia of the wheel and α is the angular acceleration. The moment of inertia can be calculated by $I_{wheel} = m_{wheel}r^2$, where m_{wheel} is the mass of the wheel. The angular acceleration can be calculated by $\alpha = \frac{a}{r}$, where a is the acceleration of the vehicle. The acceleration can be calculated by $a = \frac{F}{m}$.

The vehicle is driven by the frictional force acting on the wheel by the ground. the maximum frictional force possible is given by $F_{frict} = \mu mg$, where μ is the coefficient of friction of the tire.

The torque left to drive the vehicle is given by $\tau = \tau_{motor} - \tau_{wheel}$, where τ_{motor} is the motor torque

As such, $\tau = F_{frict}r = mar$

The motor torque required to drive the vehicle is given by:

$$\tau_{motor} = \tau + \tau_{wheel}$$

$$\tau_{motor} = mar + m_{wheel}r^2\left(\frac{a}{r}\right)$$

$$\tau_{motor} = (m + m_{wheel})ar \text{ ---(1)}$$

the maximum speed is attained when the vehicle undergoes maximum acceleration for half of the field, and maximum deceleration for the second half:

$$V_{max} = \sqrt{2as} = \sqrt{2\mu g(1.5)} = \sqrt{3\mu g}$$

It is given by $V_{max} = x(2\pi r)$, where x is the maximum rpm of the wheel and r is the radius of the wheel.

$$\text{Hence the maximum RPM of the wheel is } \frac{\sqrt{3\mu g}}{2\pi r} \text{ ---(2)}$$

The coefficient of friction is likely to be around 0.6, hence, the maximum acceleration of the vehicle would be $a = \frac{\mu mg}{m} = 0.6g$, which is $5.89ms^{-2}$. We estimate the mass of the vehicle to be 100g and the mass of the wheel to be 10g. from equation (1), the motor would have to provide up to 0.0110 Nm of torque, which is about 112 gcm^{-1} . We are using POLOLU wheels with a 34mm outer diameter, giving us a radius of 17mm. Using the same estimates, from equation (2), the motor would have to spin the wheel up to 50.8 rotations per second, which is 3047 rpm.

We chose to use a 1020 coreless motor due to its high power to weight ratio. the motor we picked is rated to spin at 13000 rpm at 12v, allowing it to spin at up to 8342 rpm at our nominal battery voltage of about 7.7v. using a 3:1 gear

reduction, it will be able to spin the wheel at 2780 rpm, which is close enough to the requirement of 3047 rpm. the motor is rated for 40 gcm^{-1} of torque, which allows it to provide up to 120 with a 3:1 gear reduction

Optimal CG location (from LATEX document)

2 longitudinal CG location

We define the CG to be at a distance of x from the rear wheels and $L-x$ from the front wheels, where L is the distance between the front and rear shafts

For the vehicle to be in equilibrium,

$$N_R + N_L = mg$$

$$N_R x = N_L (L - x)$$

$$N_R = mg \frac{L-x}{L}$$

where N_R and N_L is the normal force acting on the rear and front wheels respectively.

The vehicle is drove by the frictional force acting on the rear wheels by the ground, which is given by:

$$F_r = \mu mg$$

The maximum acceleration is given by:

$$a_{max} = \frac{F_r}{m} = \mu g \frac{L-x}{L} \quad \text{--- (3)}$$

Hence, moving the CG rearwards increases the maximum acceleration of the vehicle. This comes at the cost of handling, where a rear heavy vehicle would be more prone to understeer. To ensure predictable handling, we decided to set the CG at the center of the vehicle first, and to move it backwards if more acceleration is needed. the CG would lie exactly on the centreline of the vehicle, as there is no benefit to having a left-heavy or right-heavy vehicle.

3 Maximum CG Height

For the vehicle with wheelbase T to just avoid overtipping when navigating a turn of radius R at velocity v , the torque provided by the CG about the outer wheels is equal and opposite to the torque provided by the centrifugal force about the outer wheels:

$$mg \frac{T}{2} = \frac{mv^2}{R} h$$

$$h = \frac{Tgr}{2v^2} \quad \text{--- (2)}$$

Mass requirement

- During braking, the motor driver shorts its terminals together, turning the motor into a generator and dissipating the vehicles kinetic energy.
- A lighter vehicle would brake better as it has less kinetic energy to dissipate
- Hence we aim to keep our vehicle mass as low as possible.

Chassis sizing(from our LATEX document)

4 Chassis sizing

When the vehicle makes a turn of radius R , the centripetal force required to make it turn is provided by the frictional force acting on the wheels by the ground. if the vehicle is turning at the maximum rate, the maximum frictional force possible is exerted on the vehicle, allowing the dynamics to be described as such:

$$\begin{aligned}\mu mg &= \frac{mv^2}{R} \\ \mu &= \frac{v^2}{Rg}\end{aligned}$$

Hence the velocity of the vehicle is described by:

$$v = \sqrt{\mu Rg}$$

The time t taken to complete a 90 degree turn is given by:

$$\begin{aligned}t &= \frac{\pi R}{2v} \\ t &= \frac{\pi R}{2\sqrt{\mu Rg}} \\ t &= \frac{\pi}{2\sqrt{\mu g}}\sqrt{R} \text{ ---(1)}\end{aligned}$$

Hence the smaller the turning radius, the faster the vehicle turns. the relation between turning radius, the distance between the axles L and the width of the axle T can be found as follows:

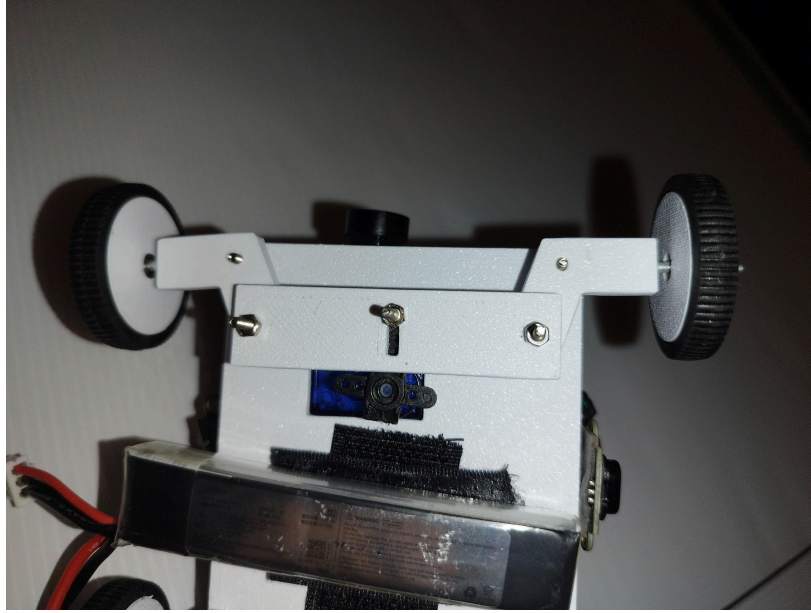
$\tan(\theta_i + \frac{\theta_o - \theta_i}{2}) = L/R$, where θ_i is the angle of the inner front wheel to the normal and θ_o is the angle of the outer front wheel to the normal. the inner front wheel is the one that is closer to the center of rotation, where the outer front wheel is the one further away.

$$\begin{aligned}R &= \frac{L}{\tan(\theta_i + \frac{\theta_o - \theta_i}{2})} \\ R &= \frac{L}{\tan(\frac{\theta_i + \theta_o}{2})} \text{ ---(2)}\end{aligned}$$

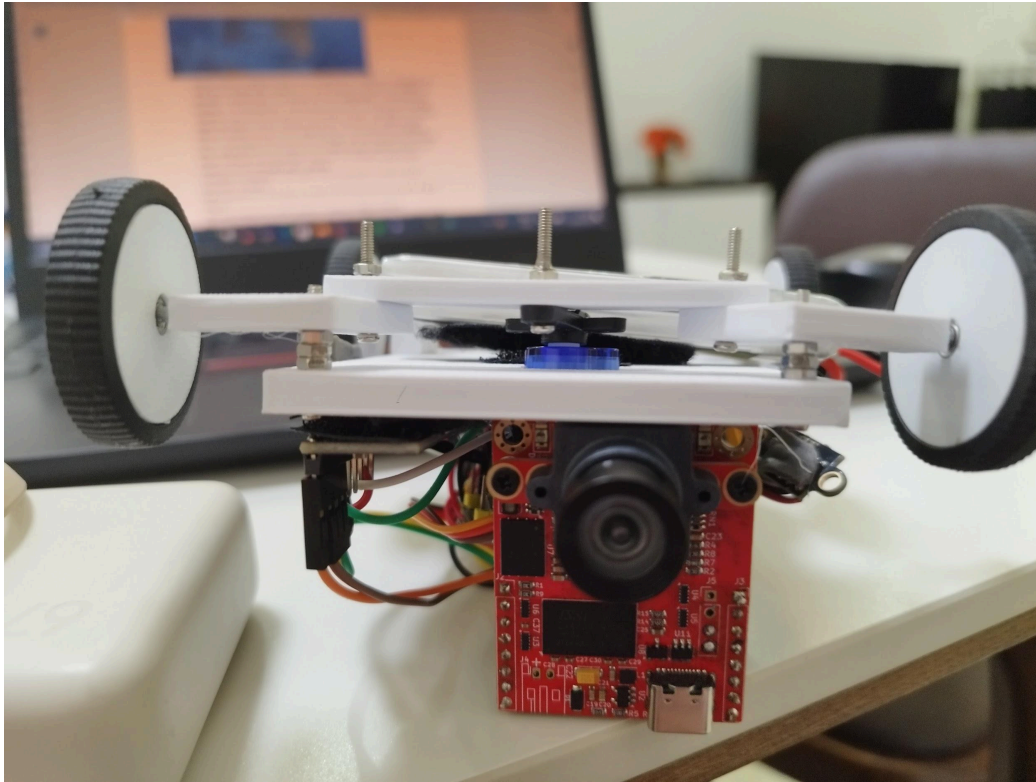
From equation (2), for the same wheel deflection, a car with a shorter distance between the axles will turn tighter and hence faster. In our chassis design, the rear axle mounts are separate from the main plate, allowing it to be mounted anywhere. This ensures that L will be as short as the battery mounting allows it to be. Since the width of our widest component, the OPENMV H7 plus, is 4cm, we set the width of the axles to be 6cm, leaving a buffer of 1cm on the left and right side.

Chassis design

- As shown in the image below, steering is done by moving the steering linkage with a servo. The servo horn is connected to the slot in the middle of the steering linkage using a M2 screw, allowing the linkage to follow along as the servo horn turns.



- As the chassis is designed before the mounting of the electronics is finalised, one of the key considerations behind our chassis design is to make it as modular as possible, so that it can accommodate as many configurations of electronics as possible. To accomplish this, we decided to mount the steering pins and linkage using M2 screws and nuts instead of 3d printing a mount. This allows us to freely adjust the distance of the steering mechanism from the main plate by using different numbers of nuts, hence allowing it to be connected to many different servos.



-
- The mounts for the rear shaft is also separate from the main plate, allowing them to be swapped out if different gears need to be used. The motor, front and rear shaft mounts, and servo are mounted using hot glue, making it easily removable with a penknife.
- The front wheels are mounted to a 2mm by 15mm metal shaft that is then inserted to the front steering arm. We cut it down to size from the 12mm by 2mm shaft used for the rear wheels
- The battery is mounted on the bottom of the main plate, to make room for electronics on the top.
- After trial and error, we ended up using 0.2mm of tolerance for all parts to get a snug fit
- We used bearings for all the wheels to minimize rolling resistance
- We used a 2mm stainless steel metal rod for the shaft because 3d printed shafts would be too weak. We used off-the-shelf plastic gears for the gear reduction because they are machined more precisely than 3d printed parts, reducing the friction between the gear teeth.
- All electronics are mounted using velcro, for easy removal

Power Management

Voltage draw

- All electronics are powered by 3.3V and 5V buck converters which step down battery voltage to 3.3V and 5V respectively
- VL53L1X must be powered off 3.3V for the logic level to be 3.3v, as the RP2040 zero is not 5V tolerant
- All other components can be powered off 5V, but we chose to power the rp2040 off 3V to free up space on the 5V rail and improve wire management
- The BNO085 is powered off the 3.3V regulator of the OpenmvH7 plus due to its proximity to it, hence freeing up space on the 3.3V rail and improving wire management

component	Input voltage tolerance	Logic level tolerance	Input voltage	Logic level
DRV8874 motor driver	4.5-37V	1.8V-5.5V input	5V and battery voltage	3.3V
Openmv H7 Plus	3.6-5V	3.3V output Input up to 5V	5V	
RP2040 Zero	2.5-3.3V (3V3 pin) 4.3-5.5V (5V/USB pin)	3.3V input and output	3.3V	
VL53L1X TOF sensor	2.6v-5.5V	3.3V-5V input and output, output level shifted to Power supply voltage	3.3V	
BNO085 IMU	3.0v-5.0V	3.3V-5V input and output, output level shifted to Power supply voltage	3.3V	
SG90 servo	4.8v-6.0V	3.3V to 5V input	5V	

Current draw

component	Maximum current draw
DRV8874 motor driver	2.1A continuous, 4.4A peak with current limiting enabled
Openmv H7 Plus	~350 mA

RP2040 zero	~100 mA (40-50mA when both ARM Cortex-M0+ cores run at 125 MHz, 50mA max from GPIO pins)
VL53L1X TOF	~20 mA continuous, 40 mA when the laser fires
BNO085	~20 mA
SG90 servo	~750 mA when stalled
TOTAL	~3.36 A continuous, ~5.70 A peak

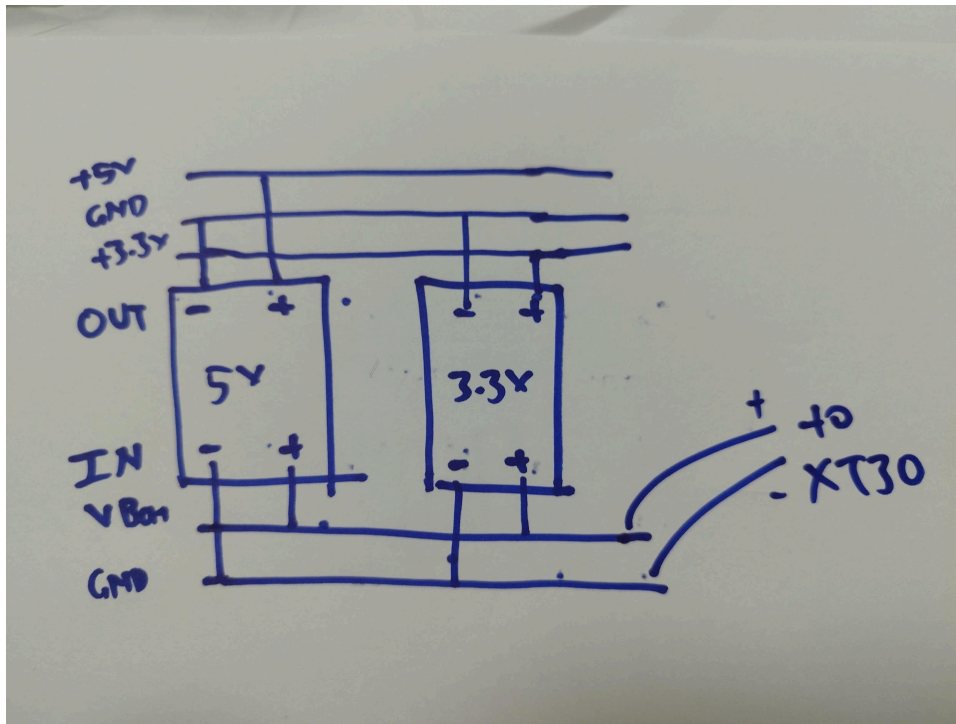
Buck converter specifications

The the 5V buck converter powers the SG90 servo and the OpenmvH7 plus, which powers the BNO085 IMU, while the 3.3V buck converter powers the RP2040 and two VL53L1X TOF sensors. The DRV8874 is powered directly by the battery. Both buck converters are able to supply the required current.

Buck converter type	Input voltage	Maximum output current	Maximum current drawn
5V	7-26V	1.8A	1.12A
3V	5.3-26V	2.5A	0.14A

Buck converter setup

- The two buck converters are soldered to a perfboard as such



- The positive input of both buck converters are soldered to the same rail, which is connected to the positive lead of the battery via an XT30 female connector
- Likewise, the negative input of both buck converters are soldered to the same rail
- The negative output of both buck converters are soldered to the same rail
- The 3.3v output and 5v output are soldered to the rails above and below the GND rail respectively

BATTERY TYPE SELECTION

The ideal battery must be:

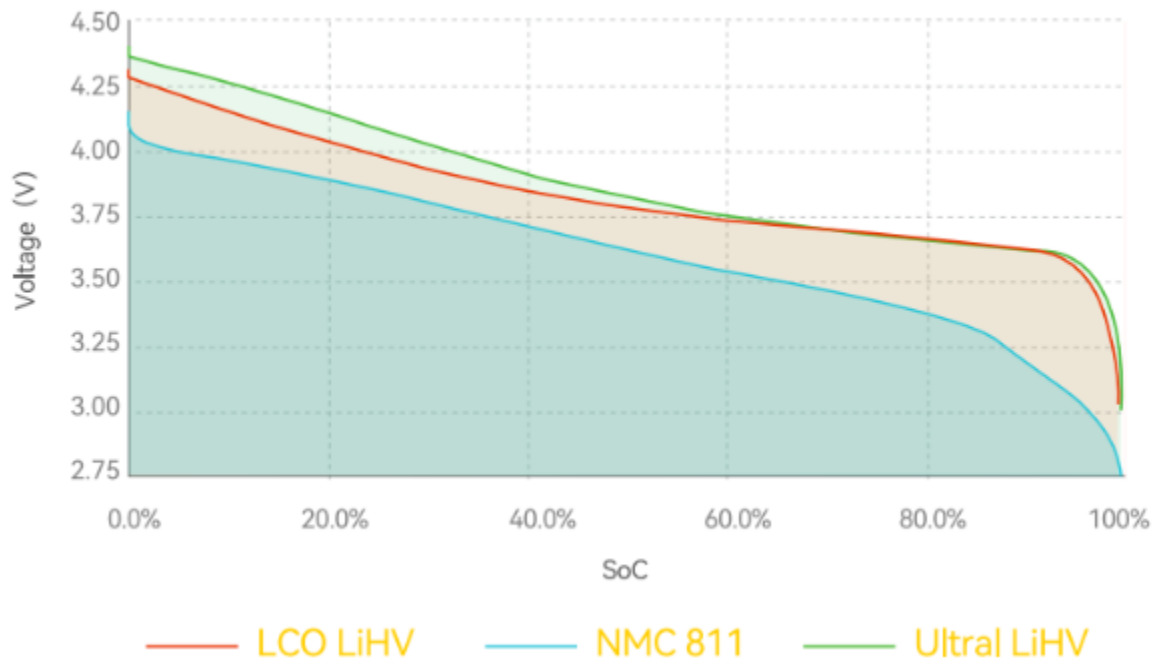
1. supply a high enough voltage and current throughout the duration of the run to be able to easily power all components on the vehicle
2. be light so total weight is reduced, allowing for faster braking and more predictable cornering
3. be thin enough so the car can sit lower to the ground, reducing the CG height and hence improving the turn radius

Hence we decided to use lithium polymer (Lipo) battery packs as they are generally lighter and smaller than lithium ion or double A battery packs of the same capacity and voltage.

Lipo batteries are also widely available, coming in a wide range of voltages, current ratings and capacities, making it easier to find one that is optimal

Characteristics of lipo batteries:

- A lipo battery has a maximum voltage of 4.2v per cell when fully charged and can be discharged down to a minimum voltage of approximately 3.0v, below which the battery becomes permanently damaged due to electrochemical effects.
- the battery C rating shows its approximate current rating: maximum continuous current = capacity (in Ah) x C rating
- as the battery discharges, the open circuit voltage per cell decreases plateaus at about 3.8 volts per cell, approximately following this graph:



- due to battery internal resistance, the terminal voltage ($v = V - RI$) will always be lower than the open circuit voltage when current is flowing
- the internal resistance increases if the system draws sustained and high current. lithium ions must move through electrolytes, separators and electrodes in order to react and supply electricity. As this process takes time, continuously drawing high current may deplete the supply of lithium electrons in certain parts of the battery, resulting in chemical disequilibrium, increasing the battery internal resistance and causing the terminal voltage to sag even more. This effect is not significant if current draw is much lower than the maximum rating, hence the importance of choosing a battery with a high C rating.

Lipo selection: Tattu R-Line 550mAh 2s

As the maximum continuous current draw is 3.36A, the vehicle would draw a maximum of 168 mAh of charge over a 3 minute run time, which is 15.3% of the total charge capacity (550mAh x 2) of the battery. This gives us an open circuit voltage of approximately 3.9V per cell at the end of the run, with a total open circuit voltage of 7.8V. Using a high estimate of internal resistance of 30mOhm and the maximum peak current of 5.70A, the maximum voltage sag would be 0.03 x

5.70 = 1.01 V. since the battery is rated for 95C of continuous current draw, which is much higher than the maximum current draw of our vehicle, we expect the internal resistance to stay constant during the run. Hence the minimum terminal voltage during the run would be about 7.7V, leaving a 0.7v buffer to the minimum input voltage of the 5v buck converter. Hence, we are confident that the Tattu R-Line 550mah 2s will be able to easily power all of our components across the duration of the run.

Electronics

Sensor architecture

- We use one microcontroller to read sensor data and another one to run the blob detection and navigation algorithms, and to control the vehicle movement. This is done to increase the speed of the control loop.
- Reading a VL53L1X sensor takes up to 20ms using the longest ranged detection mode. Reading two sensors takes anywhere from 20ms to 40ms, depending on how synchronised the two sensors are – something that varies with each run. Since sensor reading is blocking, with other parts of the code waiting for it to finish, doing this using the same microcontroller that runs blob detection and navigation code would slow down the control loop by at least 20ms.
- In comparison, if a separate microcontroller is reading sensor data, both the sensor reading and the blob detection + navigation code can run simultaneously, with the data being sent over by UART once it is collected. Since reading an approximately 40 byte data packet via a UART with 115200 baudrate only takes about 3.5 milliseconds, a dual microcontroller setup would be much faster than a single microcontroller.

Choice of components

When choosing components, we went with 2 main criteria

1. Smaller is better
 - our chassis calculations show that smaller vehicles are able to turn faster and tighter, shaving time off our runtime and improving maneuverability
2. Simpler is better
 - Based on our previous experience, a simpler system has less points of failure and is easier to debug, leading to a more reliable and repairable system

RP2040 zero

The RP2040 zero was chosen for various reasons:

- Size
 - The RP2040 zero is one of the smallest microcontrollers that has the functionality that we need. This is due to its innovative design that places pins on 3 sides of the board, while most other microcontrollers only use 2 sides
- Affordability
 - The RP2040 zero's 4 USD price is very affordable compared to alternatives from arduino and STM32, which usually cost at least 10 USD
- Functionality
 - It uses a Dual-core Arm Cortex M0+ processor, flexible clock running up to 133 MHz. This is much faster than alternatives like the arduino nano, which only runs up to 16 MHz
 - It has 29 × multi-function GPIO pins (20× via edge pinout, others via solder points), 2 × SPI, 2 × I2C, 2 × UART, 4 × 12-bit ADC, 16 × controllable PWM channels
 - This is more than our requirements of 1 UART and at least 1 I2C bus, giving us great flexibility in using additional peripherals in the future
 - Every GPIO pin can be routed to any of the 8 × Programmable I/O (PIO) state machines. PIO machines are used to quickly and precisely read the quadrature encoder used with the motor.
- Language compatibility and flexibility
 - The RP2040 zero can be programmed using c++ in the arduino IDE, something that we are already familiar with. C++ also runs faster as it is a lower level programming language.
 - It can be programmed using micropython and Thonny, which may prove useful if we decide to use micropython to keep the code simpler to read, and if its speed is sufficient. One of the team members is also much more experienced in python

Computer vision

- We decided not to use openCV because it requires a single board computer, such as a raspberry pi, running a full linux operating system to set up and use. A single board computer is not only large by itself, it also has a comparatively high power consumption, hence requiring a battery with a higher capacity. This leads to a larger chassis and a higher all up weight, reducing the speed and

maneuverability of our vehicle. An operating system, dependency management and toolchains are also needed to run opencv, making it complex to setup and use and hence prone to errors. As we only need to run simple blob detection, we do not need the advanced computer vision and machine learning algorithms provided by opencv, making it overkill for our application

- We decided to go with openmv because it can be run on cheap, low power and compact microcontrollers without the need to install operating systems, and is simple to setup using the openmv IDE. Openmv libraries also have a wide range of algorithms to detect and track simple objects, such as blob detection, line detection, and the rectangle detection that is used to find apriltags. Furthermore, the documentation is excellent, with extensive tutorials and guides.
- We decided to go with the openmvH7 plus because blob detection is relatively simple and does not require the computing power of more modern modules such as the openmv n6. The camera resolution is also greatly improved over the base H7, at 5MP instead of 0.3MP. This allows it to detect the traffic signs from further away and track the centroids of the signs more precisely, hence allowing the vehicle to navigate more precisely. This is especially important since we plan to replace the stock lens with 50 degree FOV with one that has a higher FOV of 105 degrees.
- The openmv h7 plus has 3 pins dedicated hardware PWM pins, 2 uarts and 2 I2C buses, allowing it to interface with all of the peripherals that we need.

Distance sensor

- Distance sensors are mounted to the side of our vehicle for wall detection and avoidance.
- We chose to use Time of flight (TOF) sensors instead of ultrasonic sensors because ultrasonic sensors tend to be unreliable, with fluctuating values and a tendency to malfunction and cause the i2c bus to hang. Using multiple ultrasonic sensors together will also result in interference, further degrading the readings.
- We chose to use the VL53L1X due to its long range and small form factor. Its maximum range is 400cm, which is long enough to detect the walls at the side, even after accounting for the low reflectivity of the matte black surface of the walls, which reduce the range to about 200cm
- The range of the VL53L1X is also adjustable, giving us the option to trade detection range for update rate and resistance to ambient lighting

IMU

- We chose the BNO085 because it has the lowest rotation vector drift rate, about 2 degrees per minute, out of all the options we have, even without having to use

the magnetometer. This is done using the Hillcrest Labs' proprietary SH-2 sensor fusion engine (MotionEngine)

- The sensor fusion software uses dynamic gyroscope zero-rate bias estimation to reduce gyroscope drift, and uses an extended kalman filter alongside accelerometer data to produce an accurate angular position.
- This eliminates our dependence on magnetometers, which require careful calibration in every different location, and whose reading is frequently unreliable due to the changing magnetic fields from the motor.

Motor + wheel + encoder

- We chose the POLOLU 34mm wheel because it is available in local electronics stores, making it easy to replace.
- We chose to use a 1020 coreless motor due to its high power to weight ratio and small size (see mobility management for more details)
- We hot glued an encoder designed for N20 gearmotors onto the front shaft of the coreless motor, and attached a gear to the back shaft using superglue. The encoder is mounted on the front shaft due to the motor terminals on the back blocking it.
- N20 gearmotor encoders are more common than other encoders, such as optical encoders, and are likely to have better documentation, hence the choice to use them

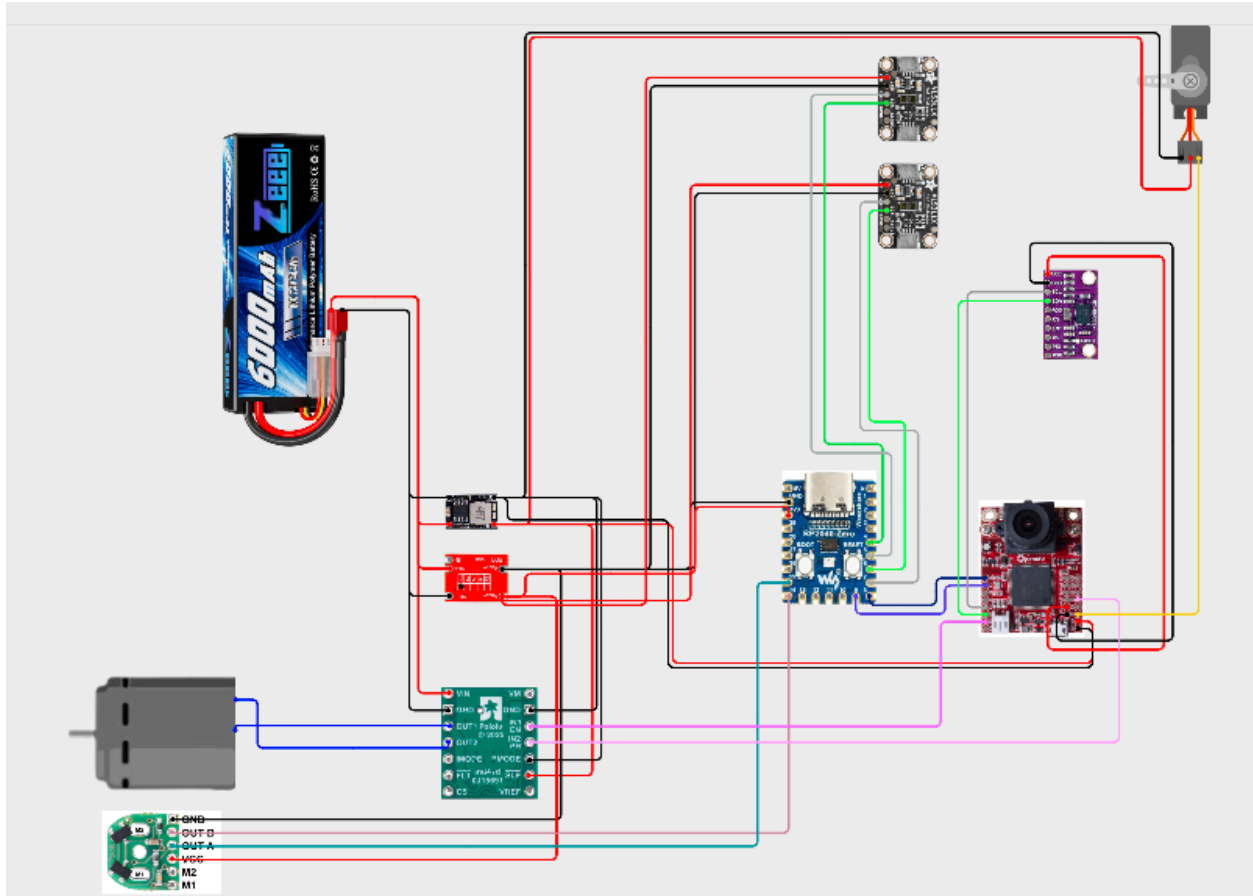
Motor driver

- The DRV8874 was chosen because its current rating is high enough to power the motor.
- It is also able to use both the PWM and PH/EN control modes, allowing for flexibility in how we control the motor.
- The documentation behind the motor driver is also extensive, with the datasheet explaining small details such as how the current limiter works

Buck converters

- We used the buck converters from our local electronics shop because it is readily available, making it easy to find replacements if we damage it during soldering, or if one of them is defective

Wiring diagram



The wiring diagram for our vehicle can be found here: [Circuit Designer IDE](#)

Rp2040 zero to VL53L1X

The VL53L1X TOF sensor communicates with the RP2040 Zero using I2C. Each sensor uses its own I2C bus instead of sharing a bus, to reduce the complexity of our code.

- TOF 1 SDA → GPIO 4 (i2c bus 1 SDA)
- TOF 1 SCL → GPIO 5 (i2c bus 1 SCL)
- TOF 2 SDA → GPIO 6 (i2c bus 2 SDA)
- TOF 2 SCL → GPIO 7 (i2c bus 2 SCL)

The RP2040 zero only has 2 I2C buses, so any additional TOFs used must share an I2C bus. Since all VL53L1X sensors share the same default address, the addresses of all the TOFs on the bus must be reassigned during startup so the I2C bus knows which sensor it is talking to. This can be done by connecting their XSHUT pins to GPIO pins, turning on the sensors one by one using and reassigning their addresses individually.

RP2040 to encoder

- The motor encoder uses quadrature encoding and sends data to the RP2040 using digital HIGH/LOW signals
- It can be read using any GPIO pins on the RP2040 zero, though most encoder libraries require the two pins to be adjacent to one another
 - OUT A → GPIO 14
 - OUT B → GPIO 15

If the encoder outputs negative values when the vehicle is moving forward, just swap the two connections.

RP2040 to Openmv H7 Plus

- The RP2040 zero communicates with the OpenmvH7 using UART. the RP2040 uses UART bus 1 (GPIO 8 and 9), and the Openmv H7 Plus uses UART bus 1 (P0, P1)
 - GPIO 8 (TX1)→ P0(RX1)
 - GPIO 9 (RX1)→ P1(TX1)

Openmv H7 Plus to DRV8874

- The Openmv H7 Plus controls the DRV8874 motor driver using Phase/enable mode instead of pwm mode. Pulling the phase pin on the DRV8874 high or low determines the direction that the motor is spinning in, while applying a PWM signal to the enable pin determines the motor speed.
Phase/enable mode is preferable to pwm mode as it has a more linear relationship between PWM duty cycle and motor speed, making it easier to tune the motor speed.
- The PH pin is connected to P9, and the EN pin is connected to P6
 - PH → P9
 - EN → P6 (PWM signal)
- NOTE: the EN pin on the motor driver and the servo signal wire should not be connected to pins on the OpenmvH7 that share the same timer (eg. P7 and P8), as the frequencies of their PWM signals are different (50hz on the servo, 20kHz on the motor driver), hence requiring separate timers.
- The motor wires are connected to the output pins
 - OUT1 → motor terminal
 - OUT2 → motor terminal
- 5V must be connected to Vsleep on the DRV8874 to set the current limiting to 4A, and GND must be connected to PMODE on the DRV8874 to enable EN/PH mode.
 - Vsleep → 5v output on buck converter
 - PMODE → GND on buck converter

Openmv H7 Plus to BNO085

- The BNO085 IMU communicates with the Openmv H7 Plus using I2C. the I2C 2 bus is used.
 - SDA → P5 (I2C 2 SDA)
 - SCL → P4 (I2C 2 SCL)

Power electronics (see power management for details)

Battery voltage

- The DRV8874 is powered off battery voltage

5V buck converter

- 5V buck converter is used to power the Openmv H7 Plus, and the SG90 servo
 - Connect all the Vin pins to the 5V rail, and all the ground pins to the ground rail
- 5V must be connected to Vsleep on the DRV8874 to set the current limiting to 4A, and GND must be connected to PMODE on the DRV8874 to enable EN/PH mode.

3.3V buck converter

- 3.3V buck converter is used to power the VL53L1X and the RP2040 Zero
 - Connect all the Vin pins to the 3.3V rail, and all the ground pins to the ground rail
- The BNO085 is powered off the 3.3v regulator on the Openmv H7 Plus

IMU calibration

- The accelerometer and gyroscope of the IMU do not require calibration on start, as they are calibrated at the factory
- The magnetometer can be calibrated by running the calibration code in src, and moving the sensor around in a figure 8 pattern until the magnetometer calibration accuracy level reaches 3.
- For our application, we do not use the magnetometer as the changing magnetic fields from the motor will affect the accuracy of the readings. As such, we take the heading from the game rotation vector output, instead of the rotation vector output on the IMU.

Color threshold calibration

- The LAB color space is used instead of RGB, as any changes in brightness would affect all 3 of the R,G,B readings, affecting the accuracy of the detection. In LAB color space, the L (lightness) term separates pure brightness from the actual chromatic color, allowing us to account for surface gloss and shadows

- Calibration is done in the OPENMV IDE threshold editor, as follows
 - Load the blob detection script onto the board → run the script to capture a image in the frame buffer → go to tools - machine vision- threshold editor → adjust the A and B sliders until the blob you are trying to detect becomes white

Obstacle management (open)

Start condition:

- Power is supplied to the vehicle by turning on a switch connected to the positive lead of the battery
- A switch connects pin P2 to ground. If the switch is closed, P2 will be pulled low and the start condition would be true. Only then would the vehicle run.

turning methods

We considered 2 main ways to detect the turn direction and turns: using the camera or the side TOF sensors:

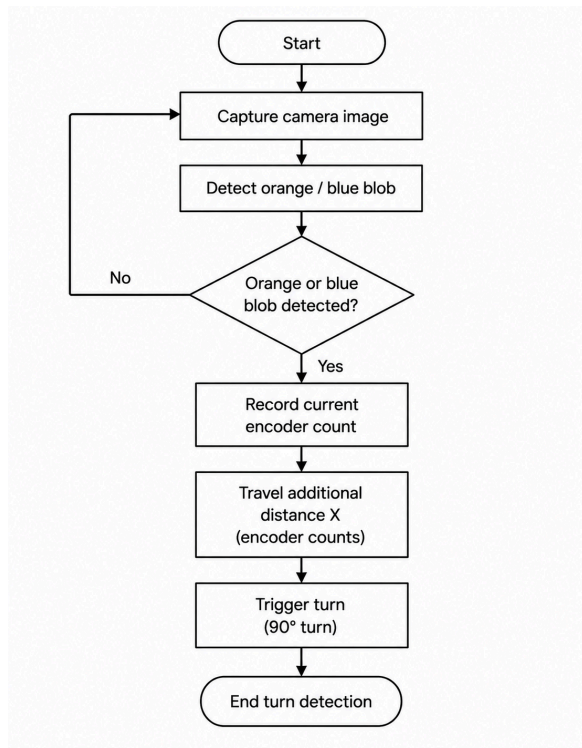
- detect turn direction
 - using camera:
 - check whether orange or blue blob is detected first

OR

- Using side TOF: check which side registers large jump in reading first
- detect turns using
 - Side TOF: turn when one side registers large jump in reading
 - Camera: turn as soon as either orange or blue blob is detected
 - May set a distance delay: turn only when an orange or blue blob is detected and the encoder increases by X counts, hence making sure the vehicle clear the inner walls, no matter the track configuration

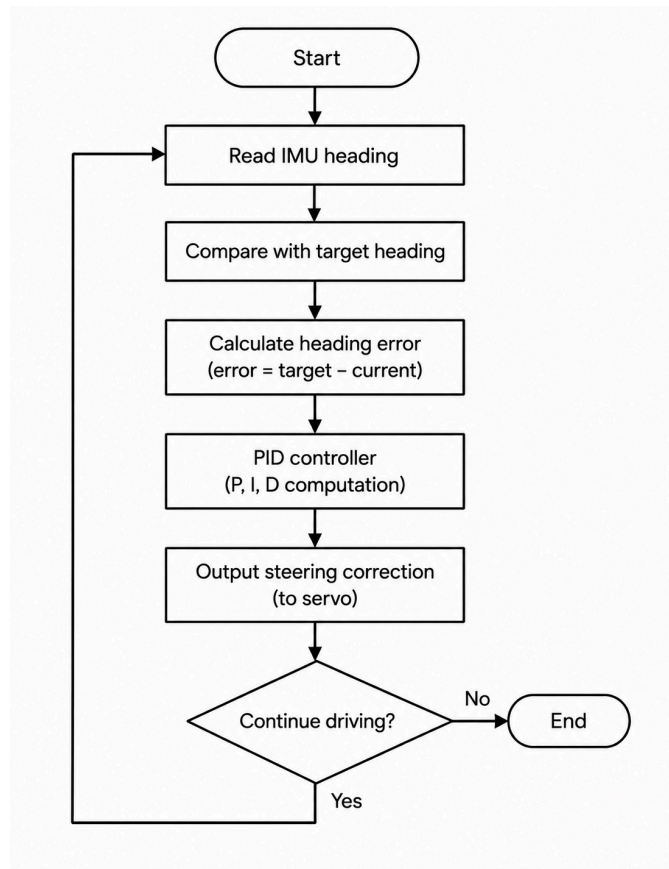
Upon further testing, we found that camera blob detection triggers turning more reliably, provided the color thresholds were tuned well.

Since a smaller radius turn also means a faster turn, as shown in the mechanical design section, we use a lower speed when turning.



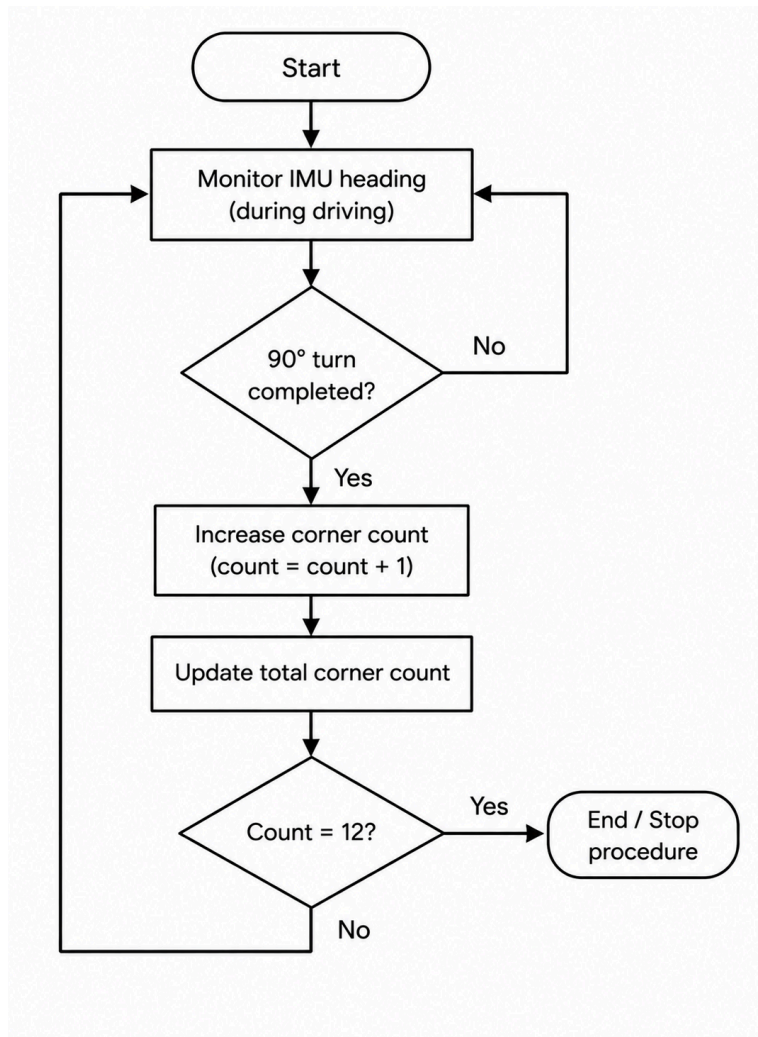
IMU PID

- Throughout the whole run, we run a PID loop using the IMU heading to ensure that the vehicle remains parallel to the walls
- If we were to hard code the turns without a PID loop, differences in wheel friction, chassis vibration and unpredictable wheel slip would cause the vehicle to fall out of alignment and crash into the walls.



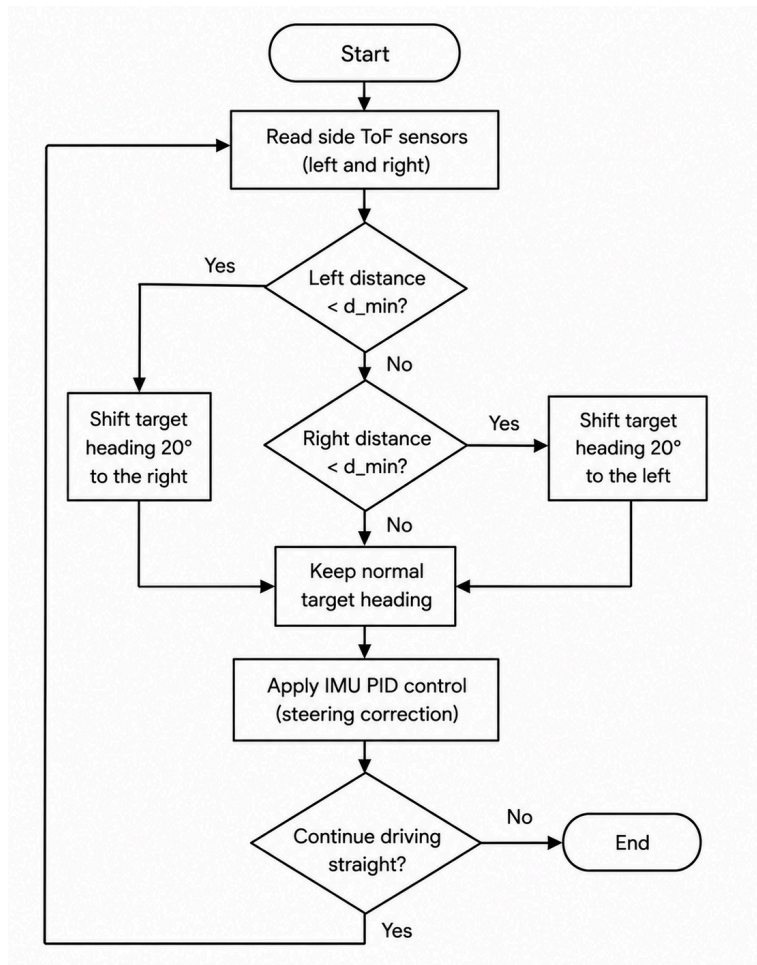
Corner counting

- We simply count the number of turns registered by the IMU



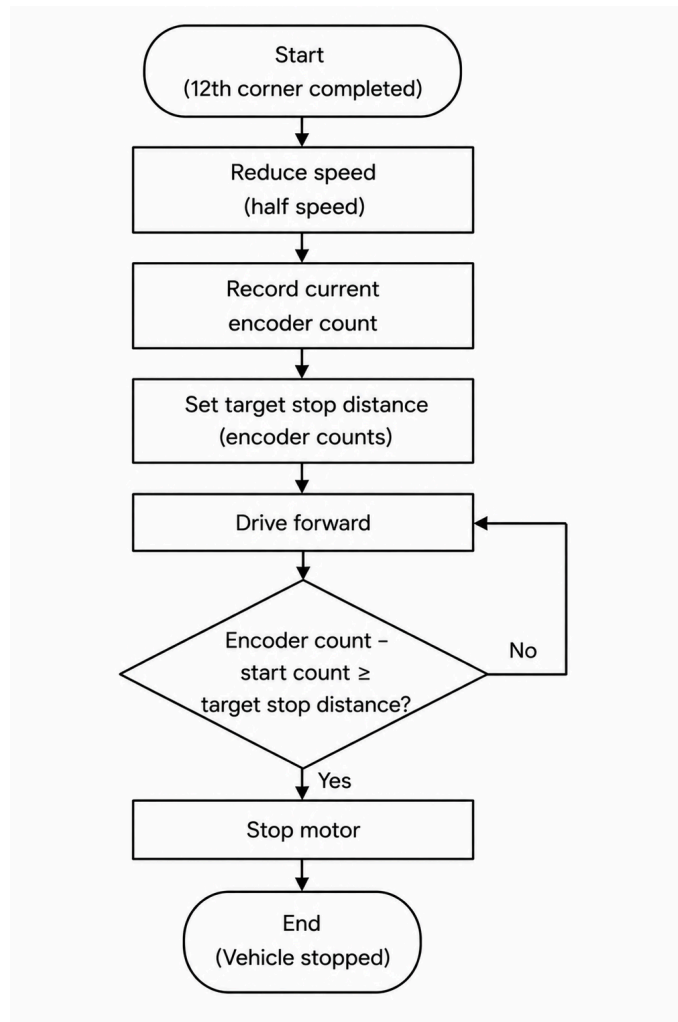
Driving straight

- Apart from the PID loop, we also use the TOF sensors for wall avoidance:
 - If the left TOF registers a reading of less than d_{min} , the setpoint heading changes by 20 degrees to the right
 - If the right TOF registers a reading of less than d_{min} , the setpoint heading changes by 20 degrees to the left
- The setpoint headings return to normal after the vehicle is more than d_{min} away from the walls
- This serves as a final failsafe if the IMU PID loop fails to keep the vehicle away from the walls

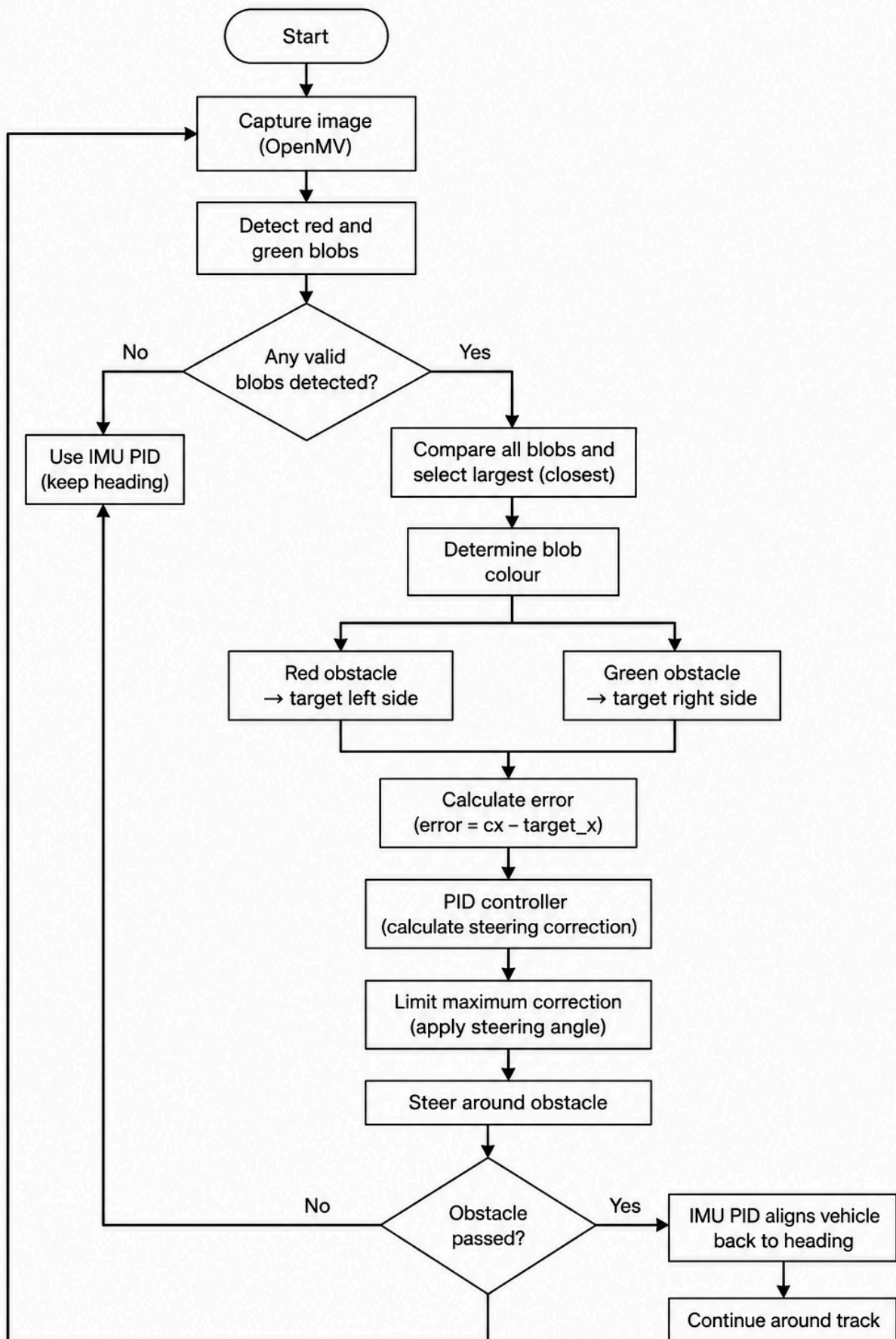


Stopping

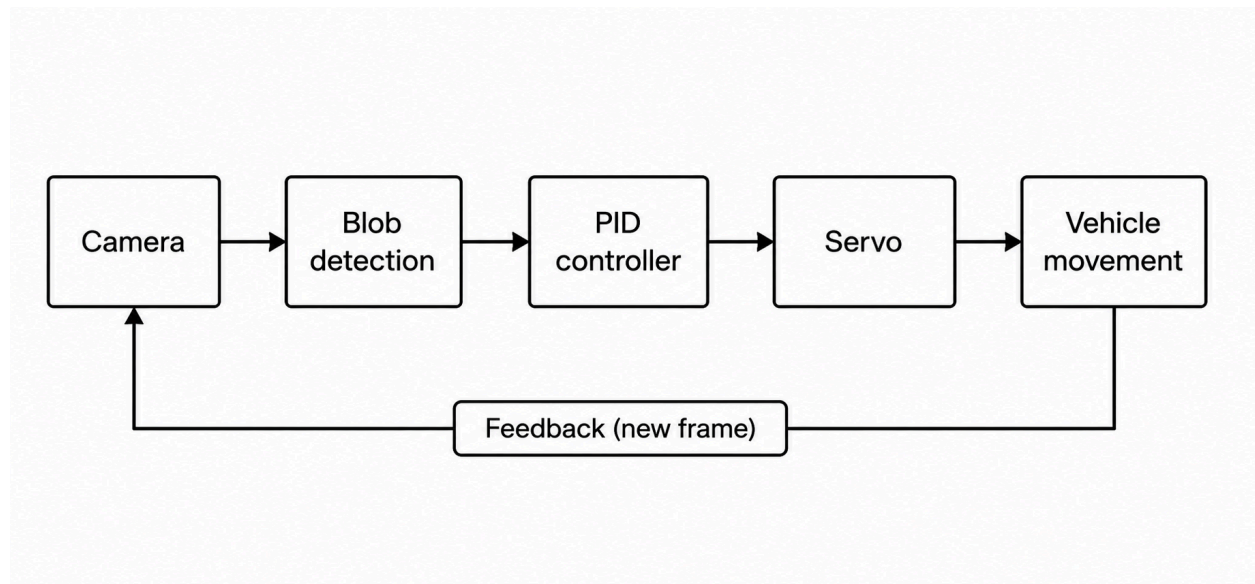
- After 12 corners are detected the vehicle comes to a stop after travelling for a certain number of encoder counts
- The speed is cut in half to ensure that the wheels do not slip, hence giving an accurate encoder reading.



Obstacle management (obstacle)



1. The red and green traffic signs are detected using the blob detection functions provided by OpenMV. Colour thresholds are used to separate red and green objects from the rest of the camera image.
2. All valid red and green blobs in the camera frame are compared. The blob with the greatest pixel area is selected, as a larger apparent area generally indicates that the traffic sign is closer to the vehicle.
3. Once an obstacle is detected, its colour determines the required passing side. For a red traffic sign, the servo direction is changed so that the block is positioned towards the left side of the camera frame, allowing the vehicle to pass the obstacle on its right. For a green traffic sign, the servo direction is changed so that the block is positioned towards the right side of the frame, allowing the vehicle to pass the obstacle on its left. The vehicle then adjusts servo direction to move back onto its original course.
4. The horizontal position of the detected blob is compared with the required target position, and the resulting error is passed into a PID controller. The controller continuously adjusts the steering servo so that the blob centroid approaches the desired position in the camera image.
5. The steering correction generated by the PID controller is limited to prevent excessively sharp steering commands. As the vehicle moves, a new camera frame is captured and the blob centroid is recalculated. This forms a closed-loop system:



The process repeats continuously instead of relying on a completely hardcoded avoidance path.

6. The relative size of the blob is also monitored. As the vehicle approaches an obstacle, the blob occupies more pixels in the image. This is done through assessing the height and width in pixels. Once pixel threshold is met, the avoidance protocol is started.
7. After the obstacle has been passed, obstacle guidance is discontinued and control returns to the IMU-based heading controller. The IMU PID controller steers the vehicle

back towards its original straight heading so that it travels approximately parallel to the track walls.

8. During turning, if another valid red or green blob is detected while the vehicle is approaching a corner or changing its direction, the vehicle returns to the obstacle-avoidance controller. A 90 degree turn can only be performed when no obstacle is detected.